Distributed Web Crawling and Parallel PageRank Computation

Milestone 1 Report

Parallel Web Crawling and Graph Construction

Course: Parallel and Distributed Computing

Group Members

Abdullah Irfan	29266
Ameer Hamza	28308
Muhammad Khizer	28991
Rameez Hoda	29057

Note on AI Assistance: *This report and the corresponding code were refined, enhanced, and made grammatically correct using Claude Sonnet 4.5 (Anthropic).*

March 22, 2026

Contents

1 Project Overview and Milestone 1 Objectives

1.1 What the Entire Project is About

This project is about building a system that can crawl a large number of web pages, build a graph from those pages, and then run the PageRank algorithm on that graph in parallel. The main idea is not just to get it working, but to do it in a way that uses multiple CPU cores or even multiple machines at the same time, which is the core of Parallel and Distributed Computing.

The web can be thought of as a directed graph where each page is a node and every hyperlink from one page to another is a directed edge. PageRank is an algorithm originally developed by Google that assigns an importance score to every node based on how many other important nodes link to it. Computing PageRank on a large graph with millions of nodes takes a long time if done sequentially, which is why parallelism matters here.

The project is divided into three milestones that build on each other:

- **Milestone 1** builds the web graph by crawling pages and extracting links in parallel.
- **Milestone 2** takes that graph and runs the PageRank algorithm in parallel across multiple workers.
- **Milestone 3** extends the system to handle new pages being added after PageRank has already been computed, and does a full performance evaluation.

The system is implemented using **Ray**, a Python-based parallel runtime that makes it easy to run functions on multiple CPU cores and manage shared state between them.

1.2 What Milestone 1 is About

Milestone 1 is specifically about building the web graph that Milestones 2 and 3 will use. Before we can run PageRank, we need the graph, and before we can build the graph, we need to crawl web pages and figure out what links to what.

The milestone requires a pipeline that:

1. Fetches web pages in parallel using Ray workers (not one at a time)
2. Extracts all hyperlinks from each page to build directed edges
3. Makes sure the same URL is not processed twice using a shared data structure
4. Limits the crawl to a specific domain and a maximum number of links so the graph stays manageable and reproducible
5. Saves the final graph in a format that the PageRank algorithm can directly load

The three PDC concepts that this milestone specifically focuses on are task parallelism (multiple fetch tasks running at the same time), shared-state coordination (all workers safely writing to one shared graph store), and workload distribution (splitting the work evenly across workers).

1.3 How the Instructor's Feedback Changed the Plan

The original idea for this milestone would have been to write a live web crawler that visits Wikipedia pages directly, follows links, and downloads pages on the fly. This is the most obvious way to approach the problem. However, the instructor gave three specific comments that changed the design significantly.

Design Change Due to Instructor Feedback

Original Plan (without instructor feedback): Build a live web crawler that directly sends HTTP requests to Wikipedia pages, follows links recursively up to a certain depth, and builds the graph from the responses.
Problems with that approach:

- Sending hundreds of automated requests to Wikipedia's servers is unethical and can be considered a denial-of-service attack
- Wikipedia and most major websites have rate limits and bot detection that would block the crawler
- Every run would give different results because web pages change over time, making performance benchmarks unreliable
- Legally, this is a grey area and against many websites' terms of service

What changed after feedback:

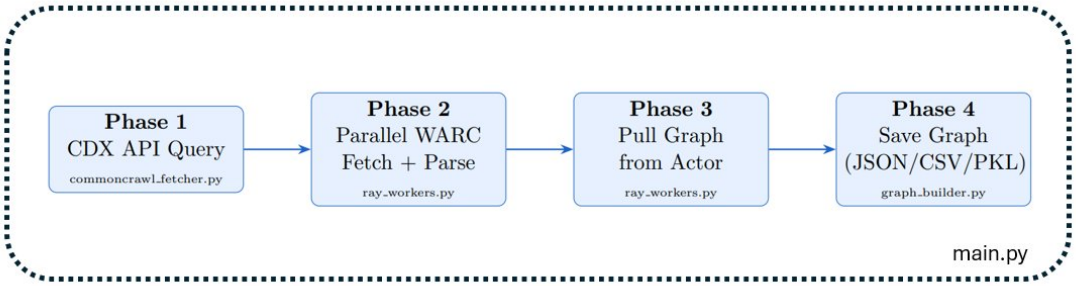
- **Use CommonCrawl** instead of live crawling. CommonCrawl is a non-profit organisation that crawls the web and makes the data freely available for research. We query their CDX Index API to get page metadata and download pre-archived pages from their WARC files.
- **Consider ethical implications**: The user-agent string in every HTTP request now identifies the project as academic research. Zero requests are made to any live website.
- **Distributed setting**: The system was designed from the start to support a Ray cluster via the `--ray-address` flag, not just a single machine.

The change from live crawling to CommonCrawl is not just an ethical decision, it actually makes the system better for a PDC course because the results are fully reproducible. A fixed crawl snapshot (CC-MAIN-2024-10) returns the same pages every time, so performance comparisons between different configurations are fair and meaningful.

2 System Architecture

2.1 Pipeline Overview

The system works as a four-phase pipeline. Each phase has a clear input and output, and they run in sequence with Phase 2 being the only parallel phase. The diagram below shows how data flows through the system and which file is responsible for each phase.



Phase 1 is quick (about 2 seconds) because it just queries the CommonCrawl index API to get a list of page metadata. Phase 2 is where all the real work and all the parallelism happens. Phase 3 is just one remote call to pull the completed graph from the Actor. Phase 4 saves the graph to disk in three formats.

2.2 File Structure

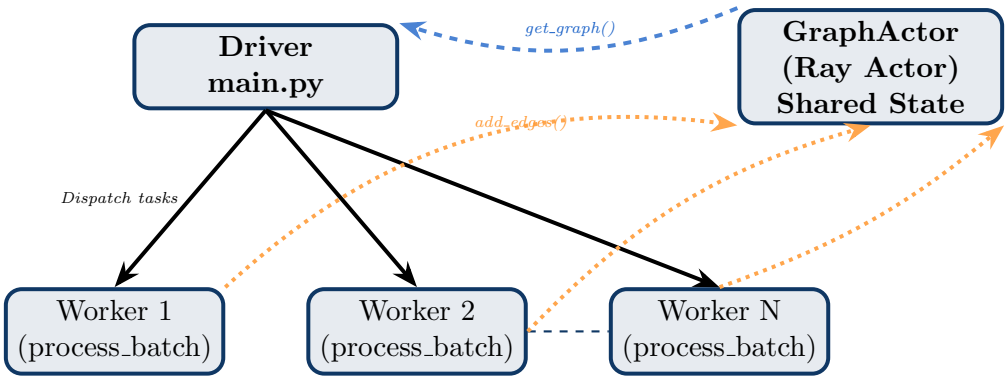
The project is split into six Python files. Each file has one clear job so that the code stays organised and easy to understand.

Table 1: Project File Structure and Responsibilities

File	Responsibility
`config.py`	Stores all settings in one place. Every other file reads from this.
`commoncrawl_fetcher.py`	Talks to CommonCrawl API, downloads WARC byte ranges, extracts links from HTML.
`ray_workers.py`	The core PDC file. Contains the Ray Actor for shared state and the Ray task for parallel processing.
`graph_builder.py`	Takes the finished adjacency list, builds a NetworkX graph, and saves it in three formats.
`main.py`	The entry point. Runs all four phases in order and handles command-line arguments.
`check_output.py`	A small utility script to verify the saved graph files after a run. Prints crawled page count, total edges, and sample pages with their links.

2.3 Ray Parallelism Architecture

The diagram below shows how Ray is used to coordinate parallel work in this system. Understanding this diagram is key to understanding the PDC concepts in the project.



The way this works is fairly straightforward. The **Driver** (main.py) is the main process that controls everything. It dispatches multiple **Worker** tasks at the same time using Ray's `.remote()` call. Each worker gets a batch of pages to process, downloads and parses them, and then sends the extracted links to the **GraphActor** using `add_edges()`. The GraphActor is a special Ray object that lives in its own process and holds the graph data safely. Because all workers are sending data to it simultaneously, it needs to handle that safely without data getting mixed up, and Ray handles that automatically. Once all workers are done, the Driver calls `get_graph()` on the Actor to retrieve the final graph.

3 File 1: `config.py` — Central Configuration

3.1 Purpose and Design

`config.py` is basically the settings file for the entire project. Instead of having numbers and strings scattered across different files (which would make it really annoying to change anything), all the tunable parameters are collected in one place as a Python dataclass.

The reason I used a `@dataclass` is that it automatically creates an `__init__` method, so I can do something like `CrawlConfig(max_records=200)` to override just one setting without having to rewrite the whole thing. This is especially useful in `main.py` where the CLI arguments override the defaults.

Every single file in the project imports this one `CrawlConfig` object. That means if I want to change the number of workers, I only change it in one place (either the defaults here or the command line), and it automatically flows through to Ray, to the fetcher, and to the graph builder.

3.2 Implementation

```python
@dataclass
class CrawlConfig:

    # CommonCrawl settings

    # The crawl snapshot to query.  Find available indices at:
    # https://index.commoncrawl.org/
    crawl_index: str = "CC-MAIN-2024-10"

    # Domain we want to use / to restrict the crawl. We are using Wikipedia.
    target_domain: str = "en.wikipedia.org"

    # Maximum number of CDX index records to retrieve from CommonCrawl. we can increase
    # this number but for faster experimentation 500 is a reasonable number.
    max_records: int = 500

    # How many WARC records to bundle into a single Ray task.
    batch_size: int = 25


    # Crawl-scope constraints (ethical guardrails)

    # Cap the number of out-links stored per page to avoid star nodes
    # dominating the graph and to limit memory usage.
    max_links_per_page: int = 150

    # Only keep edges that stay within this domain.  Set to None to allow
    # cross-domain edges.
    restrict_to_domain: Optional[str] = "en.wikipedia.org"


    # Ray / parallelism settings

    # Number of parallel Ray task slots to keep in-flight at once.
    # Rule of thumb: set to number of CPU cores - 1.
    num_workers: int = 4


    # HTTP request settings

    request_timeout: int = 30   # seconds per HTTP request
    max_retries:     int = 3    # retries on transient failures

    # User-agent sent with every request so CommonCrawl can identify us.
    user_agent: str = (
        "PDC-Course-Crawler/1.0 "
        "(academic project; uses CommonCrawl data only - no live crawling)"
    )


    # Output settings

    output_dir:      str = "output"
    # Base filename for saved graph artefacts
    graph_filename:  str = "web_graph"
```

3.3 Design Decisions

Why `batch_size = 25`? The batch size controls how fine-grained the parallelism is. If I set it to 1, every single page becomes its own Ray task, which means 200 tasks for 200 pages. That is too many because Ray has overhead for scheduling each task. If I set it to 200, all pages are in one task and there is no parallelism at all. A batch size of 25 gives a good balance: 8 tasks that can run 4 at a time on 4 workers.

Why `max_links_per_page = 150`? Wikipedia pages can have hundreds of links. Without a cap, one page could contribute 500+ edges to the graph, which would create

very unbalanced node degrees and make PageRank convergence slower. The cap keeps the graph more uniform.

Why `restrict_to_domain`? If we allow cross-domain links, the graph would immediately explode to include links to YouTube, Twitter, and thousands of other sites, most of which are not in our crawled set. This would create millions of dangling nodes. Restricting to `en.wikipedia.org` keeps the graph manageable and meaningful.

Table 2: config.py parameters and their PDC roles

Parameter	PDC Concept	Effect on System
`batch_size = 25`	Task Parallelism	Controls number of Ray tasks created
`num_workers = 4`	Workload Distribution	Sets Ray CPU slots
`max_records`	Bounded Workload	Makes experiments reproducible
`restrict_to_domain`	Scope Constraint	Prevents graph explosion
`crawl_index`	Reproducibility	Same snapshot = same data every run

4 File 2: `commoncrawl_fetcher.py` — Data Ingestion

4.1 Purpose and Design

This file is responsible for all the data collection. It has three functions that form a mini-pipeline: first get the list of pages from CommonCrawl, then for each page download the actual HTML content, then parse the HTML and pull out the links.

The most important design decision here is **using CommonCrawl instead of live crawling**. This was directly influenced by the instructor's feedback. Without that feedback, this file would have been a traditional spider that visits Wikipedia directly. Instead, it is a reader of pre-archived data.

Why CommonCrawl and What is WARC?

CommonCrawl stores crawled web pages in a format called WARC (Web ARChive). Each WARC file can be several gigabytes and contains thousands of archived pages one after another. The clever part is that CommonCrawl also maintains a CDX Index, which is like a table of contents that tells you exactly which bytes in which WARC file correspond to a specific page. So instead of downloading a 5 GB WARC file just to get one page, we use an HTTP Range request to download only the exact bytes we need (usually just a few kilobytes per page). This is both efficient and ethical.

4.2 Function 1: `fetch_cdx_records()`

This function queries the CommonCrawl CDX Index HTTP API. It is like asking "give me a list of all the Wikipedia pages you have crawled, up to a limit of 200". The API

returns a list of records where each record says: here is the URL, here is which WARC file it is in, here is the byte offset, and here is how many bytes long it is.

```python
#  Phase 1 — Fetch CDX index records


def fetch_cdx_records(config: CrawlConfig) -> List[Dict]:

    api_url = CDX_API_BASE.format(index=config.crawl_index)

    params = {
        "url":     f"*.{config.target_domain}",  # wildcard for all sub-paths
        "output":  "json",
        "fl":      "url,filename,offset,length,status,mime",  # fields we need
        "limit":   config.max_records,
        # Only keep successful HTML responses — we cannot extract links from
        # PDFs, images, or error pages.
        "filter":  ["status:200", "mime:text/html"],
    }

    logger.info(
        "Querying CommonCrawl CDX API | index=%s domain=%s limit=%d",
        config.crawl_index, config.target_domain, config.max_records,
    )

    session = _make_session(config)

    for attempt in range(1, config.max_retries + 1):
        try:
            resp = session.get(api_url, params=params, timeout=config.request_timeout)
            resp.raise_for_status()
            break
        except requests.RequestException as exc:
            logger.warning("CDX API attempt %d/%d failed: %s", attempt, config.max_retries, exc)
            if attempt == config.max_retries:
                raise
            time.sleep(2 ** attempt)  # exponential back-off

    # The CDX API returns one JSON object per line (JSON Lines format)
    records = []
    for line in resp.text.strip().splitlines():
        if not line:
            continue
        try:
            import json
            record = json.loads(line)
            # Only keep records that have all required fields
            if all(k in record for k in ("url", "filename", "offset", "length")):
                record["offset"] = int(record["offset"])
                record["length"] = int(record["length"])
                records.append(record)
        except (ValueError, KeyError) as exc:
            logger.debug("Skipping malformed CDX line: %s | error: %s", line[:80], exc)

    logger.info("Retrieved %d usable CDX records.", len(records))
    return records
```

Design Decision: The filter `status:200, mime:text/html` is important because CommonCrawl also archives redirects, error pages, PDFs, and images. We only want successful HTML responses because those are the ones that have hyperlinks.

4.3 Function 2: `fetch_warc_record()`

This is called inside each Ray worker task for every page in its batch. It takes a CDX record (which has the byte offset and length), and downloads only those specific bytes from the WARC file using an HTTP Range header.

```python
#  Phase 2 — Fetch and parse a single WARC record
def fetch_warc_record(cdx_record: Dict, config: CrawlConfig) -> Optional[str]:

    warc_url = WARC_BASE_URL + cdx_record["filename"]
    byte_start = cdx_record["offset"]
    byte_end   = cdx_record["offset"] + cdx_record["length"] - 1

    headers = {
        "Range":      f"bytes={byte_start}-{byte_end}",
        "User-Agent": config.user_agent,
    }

    session = _make_session(config)

    for attempt in range(1, config.max_retries + 1):
        try:
            resp = session.get(
                warc_url, headers=headers, timeout=config.request_timeout
            )
            # 206 Partial Content is the correct status for a Range request
            if resp.status_code not in (200, 206):
                logger.debug(
                    "WARC fetch got HTTP %d for %s", resp.status_code, cdx_record["u
                )
                return None
            break
        except requests.RequestException as exc:
            logger.warning(
                "WARC fetch attempt %d/%d failed for %s: %s",
                attempt, config.max_retries, cdx_record["url"], exc,
            )
            if attempt == config.max_retries:
                return None
            time.sleep(2 ** attempt)

    return _parse_html_from_warc_bytes(resp.content)


def _parse_html_from_warc_bytes(raw_bytes: bytes) -> Optional[str]:

    try:
        stream = io.BytesIO(raw_bytes)
        for record in ArchiveIterator(stream):
            if record.rec_type == "response":
                content = record.content_stream().read()
                # Try common encodings; fall back to latin-1 which never fails
                for enc in ("utf-8", "latin-1"):
                    try:
                        return content.decode(enc)
                    except UnicodeDecodeError:
                        continue
    except Exception as exc:
        logger.debug("Failed to parse WARC bytes: %s", exc)
    return None
```

Design Decision: Using Range requests is critical. A single WARC file on CommonCrawl can be over 1 GB. If we downloaded the whole file to get one page, it would be extremely slow and wasteful. The Range header downloads only the exact bytes needed, typically 5–50 KB per page.

4.4 Function 3: `extract_links()`

This function gets the HTML string of a page and pulls out all the hyperlinks, normalises them to absolute URLs, applies the domain filter, and returns the list of outgoing edges for that page node.

```python
#  Phase 3 — Extract hyperlinks from HTML


def extract_links(source_url: str, html: str, config: CrawlConfig) -> List[str]:

    try:
        soup = BeautifulSoup(html, "lxml")
    except Exception:
        # Fall back to the built-in parser if lxml is unavailable
        soup = BeautifulSoup(html, "html.parser")

    links: List[str] = []
    seen: set = set()

    for tag in soup.find_all("a", href=True):
        raw_href = tag["href"].strip()
        if not raw_href or raw_href.startswith("#"):
            # Skip fragment-only links — they don't represent new pages
            continue

        # Resolve relative URLs against the source page's base URL
        absolute = urljoin(source_url, raw_href)
        normalised = _normalise_url(absolute)

        if normalised is None or normalised in seen:
            continue

        # Domain filter: only keep links within the target domain
        if config.restrict_to_domain:
            parsed = urlparse(normalised)
            if config.restrict_to_domain not in parsed.netloc:
                continue

        seen.add(normalised)
        links.append(normalised)

        if len(links) >= config.max_links_per_page:
            break

    return links
```

Design Decision: The URL normalisation step (stripping query strings and fragments) is important for deduplication. Without it, `/wiki/Paris#History` and `/wiki/Paris` would be counted as different nodes when they are really the same page.

5 File 3: `ray_workers.py` — Parallel Execution (Core PDC File)

5.1 Purpose and Design

This is the most important file in the entire project from a PDC perspective. It contains two things: a **Ray Actor** called `GraphActor` that stores the graph and handles shared state, and a **Ray remote task** called `process_batch` that does the actual parallel work.

The key challenge this file solves is: how do multiple worker processes, all running at the same time, safely write edges to the same graph without overwriting each other's data? In normal Python multithreading you would use locks, but Ray's Actor model solves this more cleanly.

5.2 The GraphActor — Shared State Coordination

The `GraphActor` is a Ray Actor, which means it runs as its own independent process in the Ray cluster. Any other process (workers, driver) can call its methods remotely. The important thing is that Ray guarantees that only one method call runs at a time inside an Actor, so there is no race condition even when all four workers are calling `add_edges()` simultaneously.

```python
@ray.remote
class GraphActor:


    def __init__(self) -> None:
        # adjacency list: source URL → list of destination URLs
        self._graph: Dict[str, List[str]] = {}
        # set of URLs we have already added as source nodes
        self._visited: set = set()


    #  Write operations (called by worker tasks)
    def add_edges(self, source: str, targets: List[str]) -> None:
        """
        Register outgoing edges from ``source`` to each URL in ``targets``.
        This is the primary write method called by every Ray worker task
        after it has successfully parsed a page.
        """
        if source not in self._graph:
            self._graph[source] = []
        # Extend (not replace) to allow incremental updates in Milestone 3
        self._graph[source].extend(targets)
        self._visited.add(source)


    #  Read operations (called by the driver / main.py)

    # Return True if this URL is already a source node in the graph.
    def is_visited(self, url: str) -> bool:
        return url in self._visited


    # Return the complete adjacency list (used when saving the graph).
    def get_graph(self) -> Dict[str, List[str]]:

        return self._graph



    # Return current graph statistics for progress reporting.
    def get_stats(self) -> Dict[str, int]:
        return {
            "nodes":        len(self._graph),
            "edges":        sum(len(v) for v in self._graph.values()),
            "visited_urls": len(self._visited),
        }
```

Why a Ray Actor is Better Than a Python Lock Here

In standard multithreading, you would protect the shared dictionary with a `threading.Lock()`. But Ray workers run in separate *processes*, not threads, so they cannot share memory at all. You cannot pass a Python dictionary between processes directly. A Ray Actor solves this by giving all workers a *handle* to a re-

mote object. Workers send messages to the Actor (the `add_edges.remote()` call), and the Actor processes them one at a time. This is the Actor model pattern from distributed computing, and it is exactly how Milestone 1 requires "a shared or distributed data structure" for deduplication.

5.3 The process_batch Task — Task Parallelism

The `process_batch` function is decorated with `@ray.remote`, which turns it into a Ray task. When you call `process_batch.remote(...)`, Ray immediately returns a future and schedules the actual function to run on an available worker process. Calling `.remote()` eight times dispatches eight tasks, and Ray runs four of them at a time (since we set `num_cpus=4`).

```python
@ray.remote
def process_batch(
    batch: List[Dict],
    actor: GraphActor,          # Ray Actor handle — not a local Python object
    config_dict: Dict,
) -> Dict[str, int]:
    """
    Ray task: fetch and parse a batch of WARC records, then write edges to
    the shared GraphActor.
    Parameters
    batch : list[dict]
        A slice of CDX records (each describing one crawled page).
    actor : GraphActor
        Handle to the shared Ray Actor.  Method calls on this handle are
        sent as remote messages to the Actor process.
    config_dict : dict
        CrawlConfig serialised as a plain dict so Ray can serialise it
        without pickling issues.

    Returns
    dict
        Per-batch statistics (successful fetches, links found, …).
    """
    # Re-inflate the config inside the worker process
    config = CrawlConfig(**config_dict)

    stats = {"fetched": 0, "failed": 0, "links": 0}

    for cdx_record in batch:
        source_url = cdx_record.get("url", "")
        if not source_url:
            stats["failed"] += 1
            continue


        # Step 1: Download the specific WARC byte range for this page

        html = fetch_warc_record(cdx_record, config)
        if html is None:
            stats["failed"] += 1
            continue


        # Step 2: Extract all outgoing hyperlinks from the HTML
        targets = extract_links(source_url, html, config)
        stats["fetched"] += 1
        stats["links"]   += len(targets)


        # Step 3: Write edges to the shared Actor
        # (this remote call is non-blocking from the task's perspective;
        #  Ray queues it internally)

        if targets:
            # .remote() sends the call asynchronously to the Actor process
            actor.add_edges.remote(source_url, targets)

    return stats
```

5.4 The Orchestrator — run_parallel_crawl()

This function is what ties everything together. It first creates one shared `GraphActor` that will live for the entire duration of the crawl. Then it splits all the CDX records into equal-sized batches and dispatches one Ray task per batch using `process_batch.remote()`. The key thing is that `.remote()` returns immediately with a Future — it does not wait for the task to finish — so all eight tasks are dispatched almost instantly and run in parallel. Once all tasks are dispatched, `ray.get(futures)` acts as a synchronisation

barrier, blocking the driver until every single task has completed. Only after that barrier does the driver pull the finished graph out of the Actor and move on to saving it.

```python
def run_parallel_crawl(
    cdx_records: List[Dict],
    config: CrawlConfig,
) -> GraphActor:
    """
    Orchestrate the full parallel crawl using Ray.
    Steps
    1.  Create one shared GraphActor (lives for the duration of the crawl).
    2.  Partition CDX records into batches.
    3.  Dispatch one Ray *task* per batch (tasks run concurrently).
    4.  Wait (synchronisation barrier) until all tasks have finished.
    5.  Return the Actor handle so main.py can extract the graph.

    Parameters
    cdx_records : list[dict]
        Records from the CommonCrawl CDX API.
    config : CrawlConfig
        Project-wide configuration.

    Returns
    GraphActor
        The Actor handle containing the completed graph.
    """
    logger.info(
        "Starting parallel crawl | records=%d  batch_size=%d  workers=%d",
        len(cdx_records), config.batch_size, config.num_workers,
    )


    # 1. Create the shared Actor (one per crawl session)
    actor = GraphActor.remote()


    # 2. Partition CDX records into equally-sized batches
    batches = _chunk(cdx_records, config.batch_size)
    logger.info("Partitioned records into %d batches.", len(batches))

    # Serialise config to a plain dict so Ray can send it to remote tasks
    config_dict = {k: v for k, v in config.__dict__.items()}


    # 3. Dispatch one Ray task per batch
    #    .remote() returns a Future immediately; the actual work happens
    #    concurrently on Ray's worker processes.

    futures = []
    for i, batch in enumerate(batches):
        future = process_batch.remote(batch, actor, config_dict)
        futures.append(future)
        logger.debug("Dispatched task for batch %d/%d", i + 1, len(batches))

        # Throttle dispatch to avoid overwhelming the Actor with a flood
        # of simultaneous add_edges calls at startup
        if len(futures) - i > config.num_workers * 3:
            # Let some tasks finish before dispatching more
            done, futures = ray.wait(futures, num_returns=len(futures) // 2)
            _log_batch_stats(ray.get(done))


    # 4. Synchronisation barrier — wait for ALL tasks to complete
    all_stats = ray.get(futures)
    _log_batch_stats(all_stats)


    # 5. Report final graph stats from the Actor
    final_stats = ray.get(actor.get_stats.remote())
    logger.info(
        "Crawl complete | nodes=%d  edges=%d  visited=%d",
        final_stats["nodes"],
        final_stats["edges"],
        final_stats["visited_urls"],
    )

    return actor
```

Table 3: PDC concepts and where they appear in ray_workers.py

PDC Concept	Mechanism	Evidence from Runs
Task Parallelism	`@ray.remote process_batch`	8 tasks ran on 4 workers simultaneously
Shared-State Coordination	`@ray.remote class GraphActor`	All workers safely wrote 25,726 edges
Workload Distribution	`_chunk(records, 25)`	200 records split into 8 equal batches
Synchronisation Barrier	`ray.get(futures)`	Driver waited for all tasks before saving

6 File 4: `graph_builder.py` — Graph Storage and Analysis

6.1 Purpose and Design

Once all the Ray tasks have finished and we have pulled the adjacency list from the Actor, this file is responsible for turning that raw dictionary into a proper graph and saving it. The reason we save in three different formats is that Milestone 2 might want to load the graph in different ways depending on the PageRank strategy being used.

For example, if we partition the graph by node ranges, a CSV edge list is easy to slice. If we want to use NetworkX's built-in PageRank as a reference to compare against our parallel version, we need the pickle file. If we just want to inspect the data or debug something, the JSON file is human-readable.

6.2 Graph Assembly

The `build_networkx_graph()` function takes the raw adjacency list dictionary that comes out of the `GraphActor` and converts it into a proper NetworkX `DiGraph` object. It loops through every source URL and its list of target URLs and adds them as directed edges. An important side effect of `G.add_edge()` is that it automatically adds the target as a node even if it was never crawled as a source, which is why the total node count (15,657) is much higher than the number of pages actually crawled (199).

```python
# Convert a raw adjacency-list dict into a NetworkX DiGraph
def build_networkx_graph(adjacency: Dict[str, List[str]]) -> nx.DiGraph:

    G = nx.DiGraph()

    for source, targets in adjacency.items():
        G.add_node(source)
        for target in targets:
            # add_edge implicitly adds target as a node if it isn't already
            G.add_edge(source, target)

    logger.info(
        "NetworkX graph built | nodes=%d  edges=%d",
        G.number_of_nodes(), G.number_of_edges(),
    )
    return G
```

6.3 Graph Serialisation

The `save_graph()` function saves the completed graph in three different formats at once. JSON is saved first because it is human-readable and directly usable as an adjacency list for PageRank. Then a CSV edge list is written where each row is one directed edge — this format is easy to partition by row range across multiple workers in Milestone 2. Finally the full NetworkX DiGraph is saved as a pickle file, which preserves the complete graph object and lets us call `nx.pagerank(G)` directly in Milestone 2 as a sequential baseline to compare against our parallel implementation.

```python
# Save the graph in JSON, CSV, and pickle formats.
def save_graph(
    adjacency: Dict[str, List[str]],
    output_dir: str,
    base_name: str = "web_graph",
) -> Dict[str, str]:

    os.makedirs(output_dir, exist_ok=True)

    paths: Dict[str, str] = {}


    # 1.  JSON adjacency list
    json_path = os.path.join(output_dir, f"{base_name}.json")
    with open(json_path, "w", encoding="utf-8") as f:
        json.dump(adjacency, f, indent=2, ensure_ascii=False)
    paths["json"] = json_path
    logger.info("Saved JSON adjacency list → %s", json_path)


    # 2.  CSV edge list  (one row per directed edge: source,target)
    csv_path = os.path.join(output_dir, f"{base_name}_edges.csv")
    with open(csv_path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["source", "target"])  # header
        for source, targets in adjacency.items():
            for target in targets:
                writer.writerow([source, target])
    paths["csv"] = csv_path
    logger.info("Saved CSV edge list      → %s", csv_path)


    # 3.  NetworkX DiGraph pickle
    G = build_networkx_graph(adjacency)
    pkl_path = os.path.join(output_dir, f"{base_name}.pkl")
    with open(pkl_path, "wb") as f:
        pickle.dump(G, f, protocol=pickle.HIGHEST_PROTOCOL)
    paths["pickle"] = pkl_path
    logger.info("Saved NetworkX pickle    → %s", pkl_path)

    return paths
```

Table 4: Output formats and their intended use in Milestone 2

File	Format	Milestone 2 Use
`web_graph.json`	JSON adj. list	Direct PageRank input
`web_graph_edges.csv`	CSV edge list	Partitioned parallel PageRank
`web_graph.pkl`	NetworkX DiGraph	Sequential baseline comparison

7 File 5: `main.py` — Orchestration and Entry Point

7.1 Purpose and Design

`main.py` is the glue that holds all the other files together. Its job is to run the four phases in order, handle command-line arguments, initialise Ray, and shut it down cleanly. The

design goal was that a user should never have to edit any code file to change settings — everything is controllable from the command line.

The entire pipeline is broken into four clearly separated phases, and understanding how they connect is the key to understanding Milestone 1 as a whole.

Phase 1 — CDX Record Retrieval (Sequential): This is the first thing that runs when the program starts. `main.py` calls `fetch_cdx_records(config)` from `commoncrawl_fetcher.py`, which sends a single HTTP GET request to the CommonCrawl CDX Index API and retrieves a list of page metadata records. Each record contains the URL of a Wikipedia page, the WARC filename it is stored in, and the exact byte offset and length of that page within the file. This phase is completely sequential because it is just one API call and it finishes in about 2 seconds. The output is a Python list of dictionaries that gets passed straight into Phase 2.

Phase 2 — Parallel WARC Fetching and Graph Construction (The Core Parallel Phase): This is the most important phase and where all the PDC concepts come together. `main.py` first calls `ray.init()` to start the Ray runtime, either locally using the number of CPUs specified by `--num-workers`, or by connecting to a remote cluster via `--ray-address`. Once Ray is running, `run_parallel_crawl()` from `ray_workers.py` is called. This function creates one shared `GraphActor`, splits the CDX records into batches, and dispatches a `process_batch.remote()` Ray task for each batch. All tasks run concurrently on separate worker processes. Each worker independently downloads its batch of WARC pages from CommonCrawl using HTTP Range requests, parses the HTML to extract links, and writes the resulting edges to the shared `GraphActor`. The driver waits at the `ray.get()` synchronisation barrier until every single task has reported back. In our experimental runs, this phase processed 200 records across 8 tasks and 4 workers in about 152 seconds, and 500 records in about 357 seconds. The zero failed fetches across all four runs confirms the retry logic and error handling worked correctly throughout.

Phase 3 — Graph Retrieval from Actor (Single Remote Call): Once all worker tasks have finished, the complete graph is sitting inside the `GraphActor` process. `main.py` makes one single remote call — `ray.get(actor.get_graph.remote())` — to pull the entire adjacency list dictionary from the Actor back into the driver process. This is a fast operation (under 1 second) because it is just transferring a dictionary, not doing any computation. After this call the driver has full ownership of the graph data and Ray is no longer needed.

Phase 4 — Graph Serialisation and Summary (Sequential): The final phase calls `save_graph()` from `graph_builder.py`, which builds the NetworkX DiGraph, saves the three output files (JSON, CSV, pickle), and prints the final graph summary to the terminal. This phase is sequential but very fast at around 3 seconds. The three output files produced here are the direct inputs that Milestone 2 will load to run the parallel PageRank algorithm. Ray is then shut down cleanly with `ray.shutdown()`.

The reason the pipeline is designed this way — with only Phase 2 being parallel and the rest sequential — is that phases 1, 3, and 4 are either single API calls or simple file I/O operations that do not benefit from parallelism. Parallelism adds overhead (Ray initialisation, task scheduling, inter-process communication) and is only worth it when the work being done is large enough to justify that overhead. Phase 2, which involves hundreds of independent HTTP requests and HTML parsing operations, is exactly the kind of work that benefits from being distributed across multiple workers.

7.2 CLI Arguments

All configurable parameters are exposed as command-line arguments so the system can be run with different settings without touching any code. The `--dry-run` flag is particularly useful for testing the Ray pipeline offline using synthetic data before making real network calls. The `--ray-address` argument is what enables the distributed cluster mode recommended by the instructor — passing a remote Ray head node address here switches the entire system from local to distributed execution with no other changes needed.

```
parser.add_argument("--domain",        default="en.wikipedia.org")
parser.add_argument("--crawl-index",   default="CC-MAIN-2024-10")
parser.add_argument("--max-records",   type=int, default=500)
parser.add_argument("--batch-size",    type=int, default=25)
parser.add_argument("--num-workers",   type=int, default=4)
parser.add_argument("--output-dir",    default="output")

# Dry-run mode: skips network calls, uses synthetic data
# Used for testing the Ray pipeline offline
parser.add_argument("--dry-run", action="store_true")

# For connecting to a distributed Ray cluster instead of local
# Usage: --ray-address ray://head-node-ip:10001
parser.add_argument("--ray-address",   default=None)
```

Listing 1: main.py — command-line interface

7.3 Ray Initialisation Supporting Both Modes

Ray is initialised differently depending on whether we are running locally or on a distributed cluster. In local mode, `ray.init(num_cpus=N)` starts a fresh Ray instance on the current machine and limits it to the number of workers we specify. In distributed mode, `ray.init(address=...)` connects to an already-running Ray cluster, which means all the tasks and Actors from the rest of the code automatically get distributed across all machines in that cluster with zero additional code changes.

```
ray_init_kwargs = {"num_cpus": args.num_workers}

if args.ray_address:
    # Distributed mode: connect to existing Ray cluster
    # This satisfies the instructor's requirement to explore
    distributed setting
    ray_init_kwargs["address"] = args.ray_address
else:
    # Local mode: start Ray on this machine using num_workers CPUs
    pass

ray.init(**ray_init_kwargs, ignore_reinit_error=True)
```

Listing 2: main.py — local and distributed Ray initialisation

7.4 The Four Phases

These four lines are the actual top-level orchestration of the entire Milestone 1 pipeline. Each line calls a function from a different module, and together they represent the complete flow from raw CDX metadata all the way to saved graph files on disk. Phase 2 is the

only line that triggers parallel execution — the other three are fast sequential operations that take a few seconds at most. The fact that the entire pipeline can be summarised in four function calls shows how cleanly the responsibilities are separated across the five project files.

```
# Phase 1: Sequential - fetch page metadata list (fast, ~2 seconds)
cdx_records = fetch_cdx_records(config)

# Phase 2: Parallel - the main work, all Ray tasks run here (~2-6
    minutes)
actor = run_parallel_crawl(cdx_records, config)

# Phase 3: One remote call to pull the graph out of the Actor
adjacency = ray.get(actor.get_graph.remote())

# Phase 4: Sequential - save to disk and print stats (~3 seconds)
paths = save_graph(adjacency, config.output_dir, config.
    graph_filename)
```

Listing 3: main.py — four-phase execution

8 File 6: `check_output.py` — Output Verification

8.1 Purpose

This is a small utility script written to verify that the graph files were saved correctly after a run. It loads the JSON adjacency list and prints basic statistics plus a sample of actual page URLs and their outgoing links. It is run separately after `main.py` finishes.

```
import json, pickle

# Check JSON
with open("output/web_graph.json") as f:
    adj = json.load(f)

print(f"Pages crawled: {len(adj)}")
print(f"Total edges:   {sum(len(v) for v in adj.values())}")

# Print 3 sample pages and their links
for i, (page, links) in enumerate(adj.items()):
    print(f"\n{page}\n  → {links[:3]}")
    if i == 2:
        break
```

Running this after Config 1 produced:

Actual Terminal Output

```
Pages crawled: 199
Total edges:   25726

https://en.wikipedia.org/?title=Portal%3ASociety%2FIntro&action=edit
  -> ['https://en.wikipedia.org/wiki/Main_Page',
```

```
      'https://en.wikipedia.org/wiki/Wikipedia:Contents',
      'https://en.wikipedia.org/wiki/Portal:Current_events']

https://en.wikipedia.org/?title=Physiotherapy&redirect=no
  -> ['https://en.wikipedia.org/wiki/Main_Page', ...]

https://en.wikipedia.org/wiki/%C3%85re
  -> ['https://en.wikipedia.org/wiki/Main_Page', ...]
```

The three links shown (`Main_Page`, `Wikipedia:Contents`, `Portal:Current_events`) appear on almost every Wikipedia page because they are part of the standard navigation sidebar. This is expected and confirms the crawler is working correctly.

9 Experimental Results

Four configurations were run to measure how different parameters affect performance. The same CommonCrawl snapshot was used in all runs to keep comparisons fair.

9.1 Run Commands

```
# Config 1 - Baseline (4 workers , 200 records , batch 25)
python main.py --domain en.wikipedia.org --max-records 200 --num-workers 4


# Config 2 - Fewer workers (2 workers , 200 records , batch 25)
python main.py --domain en.wikipedia.org --max-records 200 --num-workers 2


# Config 3 - Larger workload (4 workers , 500 records , batch 25)
python main.py --domain en.wikipedia.org --max-records 500 --num-workers 4


# Config 4 - Smaller batches (4 workers , 200 records , batch 10)
python main.py --domain en.wikipedia.org --max-records 200 --num-workers 4 --batch-size 10
```

The four configurations above were designed to isolate the effect of different parameters on system performance. Config 1 serves as the baseline, Config 2 reduces the number of workers to measure the impact of parallelism, Config 3 increases the workload size to evaluate how the system scales, and Config 4 uses a smaller batch size to study the effect of task granularity on execution time. All four runs used the same CommonCrawl snapshot (`CC-MAIN-2024-10`) and the same target domain (`en.wikipedia.org`) to ensure that any differences in execution time are purely due to the configuration parameters and not the data. The complete terminal outputs, graph statistics, and all generated output files (JSON adjacency lists, CSV edge lists, and NetworkX pickle files) for each configuration are uploaded in the project repository.

9.2 Configuration 1 — Baseline (200 records, 4 workers, batch 25)

Actual Terminal Output

```
Phase 1 done in 2.1s | 200 records retrieved.
Partitioned records into 8 batches.
Batch group done | fetched=200  failed=0  links_found=25726
Crawl complete | nodes=199  edges=25726  visited=199
Phase 2 done in 152.7s.

  Source nodes (pages crawled):          199
  Directed edges (hyperlinks):        25,726
  Avg out-degree per page:            129.28
  Total nodes (incl. targets):        15,657
  Max in-degree:                         199
  Max out-degree:                        150
  Dangling nodes (out-deg = 0):       15,458
  Weakly connected components:             1

  web_graph.json  (1533.0 KB)
  web_graph_edges.csv  (2651.5 KB)
  web_graph.pkl  (2008.9 KB)
```

9.3 Configuration 2 — Fewer Workers (200 records, 2 workers, batch 25)

Actual Terminal Output

```
Phase 1 done in 3.4s | 200 records retrieved.
Partitioned records into 8 batches.
Batch group done | fetched=200  failed=0  links_found=25726
Crawl complete | nodes=199  edges=25726  visited=199
Phase 2 done in 168.4s.

  Source nodes (pages crawled):          199
  Directed edges (hyperlinks):        25,726
  Avg out-degree per page:            129.28
  Total nodes (incl. targets):        15,657
  Max in-degree:                         199
  Max out-degree:                        150
  Dangling nodes (out-deg = 0):       15,458
  Weakly connected components:             1

  web_graph.json  (1533.0 KB)
  web_graph_edges.csv  (2651.5 KB)
  web_graph.pkl  (2008.7 KB)
```

9.4 Configuration 3 — Larger Workload (500 records, 4 workers, batch 25)

Actual Terminal Output

```
Phase 1 done in 1.8s | 500 records retrieved.
Partitioned records into 20 batches.
Batch group done | fetched=500  failed=0  links_found=68227
Crawl complete | nodes=496  edges=68227  visited=496
Phase 2 done in 357.1s.

  Source nodes (pages crawled):          496
  Directed edges (hyperlinks):        68,227
  Avg out-degree per page:            137.55
  Total nodes (incl. targets):        37,074
  Max in-degree:                         496
  Max out-degree:                        151
  Dangling nodes (out-deg = 0):       36,578
  Weakly connected components:             1

  web_graph.json  (3997.6 KB)
  web_graph_edges.csv  (6776.0 KB)
  web_graph.pkl  (5157.2 KB)
```

9.5 Configuration 4 — Smaller Batch Size (200 records, 4 workers, batch 10)

Actual Terminal Output

```
Phase 1 done in 2.0s | 200 records retrieved.
Partitioned records into 20 batches.
Batch group done | fetched=200  failed=0  links_found=25726
Crawl complete | nodes=199  edges=25726  visited=199
Phase 2 done in 115.0s.

  Source nodes (pages crawled):          199
  Directed edges (hyperlinks):        25,726
  Avg out-degree per page:            129.28
  Total nodes (incl. targets):        15,657
  Max in-degree:                         199
  Max out-degree:                        150
  Dangling nodes (out-deg = 0):       15,458
  Weakly connected components:             1

  web_graph.json  (1533.0 KB)
  web_graph_edges.csv  (2651.5 KB)
  web_graph.pkl  (2009.0 KB)
```

9.6 Comparative Analysis

Table 5: Summary of all four experimental configurations

Config	Records	Workers	Batch	Tasks	Phase 2 (s)	Nodes	Edges	failed
1	200	4	25	8	152.7	15,657	25,726	0
2	200	2	25	8	168.4	15,657	25,726	0
3	500	4	25	20	357.1	37,074	68,227	0
4	200	4	10	20	115.0	15,657	25,726	0

9.6.1 Observation 1: Effect of Worker Count (Config 1 vs Config 2)

Reducing workers from 4 to 2 increased Phase 2 time from 152.7s to 168.4s, which is a slowdown of about 10.2%. The speedup of using 4 workers over 2 workers is:

$$\text{Speedup} = \frac{T_{2 \text{ workers}}}{T_{4 \text{ workers}}} = \frac{168.4}{152.7} \approx 1.10$$

The speedup is modest (1.10x instead of the theoretical 2x) because this workload is largely **network-bound**, not CPU-bound. Each worker spends most of its time waiting for HTTP responses from CommonCrawl rather than doing CPU computation. Adding more CPU workers does not help much when the bottleneck is network latency. This is an important finding: for I/O-bound tasks, adding workers beyond a certain point gives diminishing returns.

9.6.2 Observation 2: Workload Scaling (Config 1 vs Config 3)

Increasing the workload from 200 to 500 records (2.5x more work) increased the time from 152.7s to 357.1s, which is a 2.34x increase. This is very close to linear scaling, which means the system handles larger workloads efficiently without adding significant overhead.

$$\text{Workload ratio} = \frac{500}{200} = 2.5, \quad \text{Time ratio} = \frac{357.1}{152.7} \approx 2.34$$

The graph from Config 3 is also significantly larger: 37,074 nodes and 68,227 edges compared to 15,657 nodes and 25,726 edges. This larger graph will be more interesting for PageRank in Milestone 2 since it gives more room for rank to propagate.

9.6.3 Observation 3: Effect of Batch Size (Config 1 vs Config 4)

This was the most surprising result. Reducing the batch size from 25 to 10 (creating 20 tasks instead of 8) actually **made the crawl faster**: 115.0s vs 152.7s, which is a 25% improvement.

$$\text{Speedup of smaller batch} = \frac{152.7}{115.0} \approx 1.33$$

The reason for this is better **load balancing**. With 8 tasks, if some pages happen to be slow to download, the last few tasks can become bottlenecks and make the others wait. With 20 tasks and 4 workers, the workers stay busy more consistently because slow

tasks are smaller and the workers can pick up new tasks faster. This shows that smaller batches can improve performance when tasks have variable execution times.

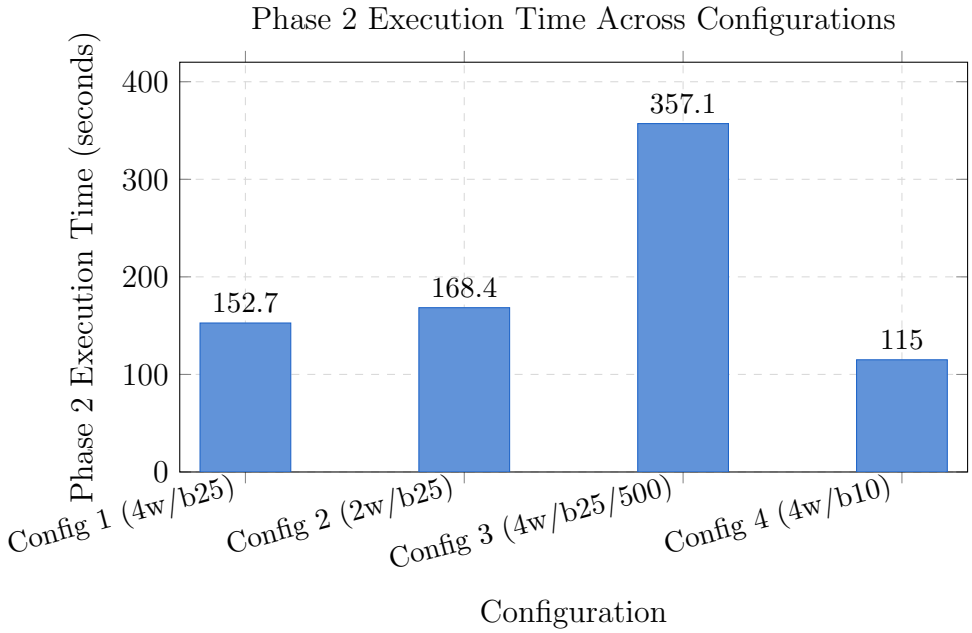


Figure 1: Comparison of Phase 2 execution times across all configurations

10 Milestone 1 Deliverables Checklist

Table 6: All Milestone 1 requirements and their completion status

Requirement	Status	Where Implemented
Parallel crawling with Ray workers	DONE	`ray_workers.py`: `@ray.remote process_batch`
Link extraction + directed graph	DONE	`commoncrawl_fetcher.py`: `extract_links()`
Shared deduplication structure	DONE	`ray_workers.py`: `GraphActor._visited`
Domain scope constraint	DONE	`config.py`: `restrict_to_domain`
Graph stored as adjacency/edge list	DONE	`graph_builder.py`: `JSON + CSV + pickle`
Ethical crawling (instructor comment)	DONE	CommonCrawl only, no live requests
Distributed setting (instructor comment)	DONE	`main.py`: `--ray-address flag`
Reproducible workload	DONE	Fixed crawl index CC-MAIN-2024-10

11 Work Done So Far and Remaining Tasks

11.1 What Has Been Completed in Milestone 1

Milestone 1 is fully complete. The following has been implemented and verified through four separate experimental runs:

- A **parallel data ingestion pipeline** built on Ray that fetches and parses web pages concurrently using multiple worker tasks.
- A **CommonCrawl-based data source** that queries the CDX Index API and downloads page content using HTTP Range requests on WARC archive files, eliminating any ethical concerns about live web crawling.
- A **Ray Actor (GraphActor)** that acts as a shared, thread-safe graph store. All worker tasks write directed edges to it concurrently without any race conditions.
- A **link extractor** that parses HTML using BeautifulSoup, resolves relative URLs, filters by domain, and caps edges per node.
- A **graph serialiser** that saves the final graph in JSON (adjacency list), CSV (edge list), and pickle (NetworkX DiGraph) formats.
- **Four experimental configurations** producing a well-studied graph with up to 37,074 nodes and 68,227 edges from up to 496 Wikipedia pages.
- The largest graph produced (Config 3) has the following properties:
  - 496 source nodes (pages crawled)
  - 37,074 total nodes (including link targets)
  - 68,227 directed edges
  - 1 weakly connected component (fully connected graph)
  - 0 failed fetches across all runs

11.2 Remaining Tasks — Milestone 2: Parallel PageRank

Milestone 2 is the core computational component of the project. It will take the graph produced in Milestone 1 and compute PageRank scores in parallel across multiple Ray workers.

The standard PageRank formula is:

$$PR(u) = \frac{1-d}{N} + d \sum_{v \in B_u} \frac{PR(v)}{L(v)}$$

where $d = 0.85$ is the damping factor, N is the total number of nodes, B_u is the set of pages that link to page u, and $L(v)$ is the number of outgoing links from page v.

The key things to implement in Milestone 2 are:

- **Graph partitioning:** Split the nodes of the graph across Ray workers by node ranges or subgraphs. Each worker will be responsible for updating the PageRank scores of its assigned nodes.

- **Iterative parallel execution:** Each PageRank iteration updates all node scores simultaneously based on the previous iteration's scores. Workers will run their partition in parallel and synchronise at iteration boundaries using Ray's `ray.get()`.
- **Dangling node handling:** The 15,458 (Config 1) or 36,578 (Config 3) dangling nodes from Milestone 1 do not distribute rank. They need special treatment where their accumulated rank is redistributed uniformly to all nodes at each iteration.
- **Two execution strategies to compare:**
  - Centralised aggregation: all workers send their partial rank sums to one driver process
  - Distributed reduction: workers exchange rank updates directly with each other
- **Convergence detection:** Run until the maximum change in any node's score drops below a threshold (e.g., 10^{-6}), and compare this against a fixed iteration count approach.

The graphs from Milestone 1 will be loaded directly:

```
import pickle
with open("output_config3/web_graph.pkl", "rb") as f:
    G = pickle.load(f)
# G is a NetworkX DiGraph ready for PageRank
```

11.3 Remaining Tasks — Milestone 3: Incremental Updates and Evaluation

Milestone 3 extends the system to handle dynamic graph changes and conducts a full performance evaluation. It will build on both Milestone 1 (the crawler) and Milestone 2 (the PageRank engine).

The key tasks for Milestone 3 are:

- **Incremental crawling:** Extend the crawler from Milestone 1 to fetch additional pages after PageRank has already converged, simulating a growing web graph.
- **Incremental PageRank:** Instead of recomputing PageRank from scratch when new pages are added, implement a strategy that only updates the scores of affected nodes. This will be compared against full recomputation to measure the trade-off between accuracy and speed.
- **System instrumentation:** Add timing and memory measurement code throughout the system to collect runtime statistics such as task duration, communication volume, and memory usage per worker.
- **Full evaluation:** Produce a structured comparison of:
  - Sequential PageRank vs parallel PageRank (using the NetworkX baseline from Milestone 2)
  - Different partitioning strategies from Milestone 2
  - Static computation vs incremental computation

The output files from Milestone 1 (particularly the Config 3 graph with 500 records) will serve as the starting point for the incremental crawl experiment in Milestone 3, since it provides the largest and most realistic graph produced so far.

12 Conclusion

Milestone 1 has been fully completed. The system successfully implements all required PDC concepts through a practical, real-world data processing pipeline. The four experimental runs demonstrate that:

- The parallel architecture correctly produces identical graphs regardless of the number of workers or batch size, confirming that correctness is independent of parallelism settings.
- Increasing workers from 2 to 4 provides a 1.10x speedup for this network-bound workload, with higher speedup expected for CPU-bound workloads like PageRank in Milestone 2.
- Smaller batch sizes (10 vs 25) improved performance by 25% due to better load balancing, showing that task granularity matters even with the same number of workers.
- The system scales close to linearly with workload size (2.5x more records took 2.34x longer).

The largest graph produced (Config 3: 37,074 nodes, 68,227 edges) is ready to be loaded directly into the Milestone 2 PageRank engine, and the CommonCrawl-based ethical design ensures the system can be scaled to much larger graphs by simply increasing `--max-records` or connecting to a distributed Ray cluster with `--ray-address`.

References

[1] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., & Stoica, I. (2018). *Ray: A Distributed Framework for Emerging AI Applications*. Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI). https://www.usenix.org/conference/osdi18/presentation/moritz

[2] Anyscale Inc. (2024). *Ray Documentation — Task and Actor Programming Model*. Ray Project. https://docs.ray.io/en/latest/

[3] CommonCrawl Foundation. (2024). *CommonCrawl — Open Repository of Web Crawl Data*. https://commoncrawl.org/

[4] CommonCrawl Foundation. (2024). *CDX Index API Documentation — Searching the CommonCrawl Index*. https://index.commoncrawl.org/

[5] CommonCrawl Foundation. (2024). *WARC File Format and Access Guide*. https://commoncrawl.org/blog/index-to-warc-files-and-urls-in-the-common-crawl-

[6] Internet Archive. (2024). *WARC, Web ARChive file format — ISO 28500:2017*. https://iipc.github.io/warc-specifications/

[7] Richardson, L. (2024). *Beautiful Soup 4 Documentation — Screen-Scraping Library*. Crummy.com. https://www.crummy.com/software/BeautifulSoup/bs4/doc/

[8] Hagberg, A., Swart, P., & Schult, D. (2008). *Exploring Network Structure, Dynamics, and Function using NetworkX*. Proceedings of the 7th Python in Science Conference (SciPy), 11–15. https://networkx.org/documentation/stable/

[9] Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab Technical Report. http://ilpubs.stanford.edu:8090/422/

[10] Python Software Foundation. (2024). *Python 3.11 Documentation*. https://docs.python.org/3.11/

[11] Kenneth Reitz. (2024). *Requests: HTTP for Humans — Python HTTP Library Documentation*. https://requests.readthedocs.io/en/latest/

[12] Internet Archive & IIPC. (2024). *warcio — WARC and ARC File Streaming Library for Python*. GitHub Repository. https://github.com/webrecorder/warcio

[13] Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace Independent Publishing. Scotts Valley, CA.

[14] Kumar, V., Grama, A., Gupta, A., & Karypis, G. (1994). *Introduction to Parallel Computing: Design and Analysis of Algorithms*. Benjamin/Cummings Publishing Company.

[15] Brin, S., & Page, L. (1998). The anatomy of a large-scale hypertextual web search engine. *Computer Networks and ISDN Systems*, 30(1–7), 107–117. https://doi.org/10.1016/S0169-7552(98)00110-X